HOMEWORK 3

Problem 4

```python
def floyd_warshall_with_path(self):  1 usage
    n = self.get_num_vertices()
    INF = float('inf')

    # Initialize the distance (dist) and path (prev) matrices
    dist = [[INF] * n for _ in range(n)]
    prev = [[-1] * n for _ in range(n)]  # f[i][j] will store the next vertex to go from i to j

    # Initialize distances and paths based on the graph's edges
    for i in range(n):
        for j in range(n):
            if i == j:
                dist[i][j] = 0  # the distance from a vertex to itself is 0
                prev[i][j] = j
            elif (i, j) in self.edge_costs:  # if there is an edge from i to j
                dist[i][j] = self.edge_costs[(i, j)]  # set the edge's cost
                prev[i][j] = i  # the next vertex on the shortest path is j

    # Perform the standard Floyd-Warshall algorithm to compute the shortest paths
    for k in range(n):
        # Consider k as the intermediate vertex
        for i in range(n):
            for j in range(n):
                # Check if a shorter path exists using the intermediate vertex k
                if dist[i][j] > dist[i][k] + dist[k][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
                    prev[i][j] = prev[k][j]  # update the path to go through vertex k

    return dist, prev
```

```python
def reconstruct_path(self, u, v,prev):  1 usage
    # if there is no path from u to v
    if prev[u][v] == -1:
        return []

    # reconstruct the path from u to v using the f matrix
    path = [v]
    while u != v:
        v = prev[u][v]
        path.append(v)

    return path

def find_lowest_cost_walk(self, start, end):  1 usage (1 dynamic)
    # run Floyd-Warshall to get the distance and path matrices
    w, f = self.floyd_warshall_with_path()

    INF = float('inf')

    # Check if there's no path
    if w[start][end] == INF:
        return False, "NO_PATH"

    path = self.reconstruct_path(start, end, f)

    # return the result along with the minimum cost
    min_cost = w[start][end]
    return True, (path, min_cost)
```

UI:

```python
    if ok == 1:
        try:
            start = int(input("Enter start vertex: "))
            end = int(input("Enter end vertex: "))

            success, data = self.graph.find_lowest_cost_walk(start, end)

            if success:
                path, min_cost = data
                path.reverse()
                print(f"Path: {' -> '.join(map(str, path))}")
                print(f"Minimum cost: {min_cost}")
            else:
                if data == "NO_PATH":
                    print("Error: No path exists between the vertices.")
                elif data == "NEGATIVE_CYCLE":
                    print("Error: The graph contains a negative cycle.")
        except ValueError:
            print("Invalid input. Please enter integer values for vertices.")
    else:
        print("No graph loaded.")
```

Bonus 1:

```python
def bonus1(self, start, end):  1 usage (1 dynamic)
    from heapq import heappop, heappush # for the priority queue to manage the minimum costs
    from collections import defaultdict # for default dictionary for path count management

    # initialize the priority queue with the start vertex and cost 0
    priority_queue = []
    heappush(priority_queue, item: (0, start))

    # default distance for each vertex is infinity, except for the start vertex
    distances = defaultdict(lambda: float('inf'))
    distances[start] = 0

    # initialize path count for each vertex, starting with 1 path to the start
    path_count = defaultdict(int)
    path_count[start] = 1

    while priority_queue:
        current_cost, current_vertex = heappop(priority_queue) # get the vertex with the smallest cost
        # if the current vertex is the destination, skip further processing
        if current_vertex == end:
            continue

        # explore all the neighbors of the current vertex
        for neighbor, _ in self.get_out_edges(current_vertex):
            edge_cost = self.edge_costs[(current_vertex, neighbor)]
            new_cost = current_cost + edge_cost # calculate the new cost to reach this neighbor

            # if we found a cheaper way to reach this neighbor, update its distance and path count
            if new_cost < distances[neighbor]:
                distances[neighbor] = new_cost
                path_count[neighbor] = path_count[current_vertex]   # inherit the number of paths to the current vertex
                heappush(priority_queue, item: (new_cost, neighbor))
            # if we found an equally cheap way, increase the number of paths to this neighbor
            elif new_cost == distances[neighbor]:
                path_count[neighbor] += path_count[current_vertex]

    return distances[end], path_count[end] if end in path_count else 0
```

UI:

```python
def min_cost_path_25(self):  1 usage
    source = int(input("Enter the source vertex: "))
    destination = int(input("Enter the destination vertex: "))
    min_cost, count_paths = self.graph.bonus1(source, destination)
    if count_paths == 0:
        print(f"No paths available from {source} to {destination}.")
    else:
        print(f"Minimum cost from {source} to {destination} is {min_cost} with {count_paths} distinct path(s).")
```

Bonus 2:

```python
# performs a topological sort on the graph (for DAG)
def topological_sorting(self):  1 usage
    from collections import deque
    # initialize in-degree dictionary to keep track of how many edges point to each vertex
    inbound_degree = {i: 0 for i in range(self.num_vertices)}
    # update in-degree of each vertex by iterating over its outgoing edges
    for vertex in self.get_vertices():
        for neighbor,_ in self.get_out_edges(vertex):
            inbound_degree[neighbor] += 1  # increase in-degree for each neighbor
    # initialize the queue with all vertices that have no incoming edges
    queue = deque([v for v in inbound_degree if inbound_degree[v] == 0])
    topological_order = []
    # process the queue to generate the topological order
    while queue:
        vertex = queue.popleft()    # get the vertex with no incoming edges
        topological_order.append(vertex)    # add it to the topological order
        # decrease the in-degree of all its neighbors, and if any neighbor has no incoming edges, add it to the queue
        for neighbor,_ in self.get_out_edges(vertex):
            inbound_degree[neighbor] -= 1
            if inbound_degree[neighbor] == 0:
                queue.append(neighbor)

    if len(topological_order) != self.num_vertices:
        return None
    return topological_order
```

```python
def count_paths(self, start, end):  1 usage (1 dynamic)
    topological_order = self.topological_sorting()
    if topological_order is None:
        return 0    # if the graph has a cycle, return 0 paths
    # initialize an array to store the number of paths to each vertex, starting with 1 path to the start vertex
    path_count = [0] * self.num_vertices
    path_count[start] = 1

    for vertex in topological_order:
        if path_count[vertex] > 0:
            # update the path count for all its neighbors
            for neighbor,_ in self.get_out_edges(vertex):
                path_count[neighbor] += path_count[vertex]  # add the paths from this vertex to its neighbors

    return path_count[end]
```

```python
def count_paths(self):  2 usages (1 dynamic)
    source = int(input("Enter the source vertex: "))
    destination = int(input("Enter the destination vertex: "))
    num_paths = self.graph.count_paths(source, destination)
    print(f"The number of distinct walks from vertex {source} to vertex {destination} is {num_paths}.")
```

Bonus 3:

```python
from graph import DirectedGraph

from itertools import combinations

class BridgeGraph(DirectedGraph):  1 usage
    def __init__(self, crossing_times):
        # initializes the graph with the number of people as vertices
        super().__init__(len(crossing_times))
        self.cross_time = crossing_times
        self.vertices = []

    def generate_all_states(self):  1 usage
        # generates all possible states of people on one side and the position of the torch
        people = set(range(len(self.cross_time)))
        for count in range(len(self.cross_time) + 1):
            for people in combinations(people, count):
                # append states for both torch positions (True for original side, False for the other side)
                self.vertices.append((frozenset(people), True))
                self.vertices.append((frozenset(people), False))

    def get_neighbors(self, state):  1 usage
        # given a state, returns all possible next states (neighbors) and the cost of moving to those states
        current_side, torch_pos = state
        opposite_side = set(range(len(self.cross_time))) - current_side

        neighbors = []
        if torch_pos:
            # if the torch is on the original side, combinations of 2 people can cross
            moves = combinations(current_side, r: 2) if len(current_side) > 1 else [(p,) for p in current_side]
        else:
            # if the torch is on the other side, only one person can return with the torch
            moves = combinations(opposite_side, r: 1)

        # iterate over all possible moves
        for move in moves:
            # calculate the new state after the move
            new_side = set(current_side).symmetric_difference(move)
            # the cost of the move is determined by the slowest person
            move_cost = max(self.cross_time[p] for p in move)
            # add the new state to the neighbors list with the corresponding move cost
            new_state = (frozenset(new_side), not torch_pos)   # switch the torch position
```

```python
            new_state = (frozenset(new_side), not torch_pos)   # switch the torch position
            neighbors.append((new_state, move_cost))

        return neighbors

    def solution(self):  1 usage (1 dynamic)
        from heapq import heappush, heappop
        priority_queue = []
        start_state = (frozenset(range(len(self.cross_time))), True)
        heappush(priority_queue, item: (0, start_state))
        visited = {}
        while priority_queue:
            # get the state with the lowest cost
            cost, state = heappop(priority_queue)
            # check if we have reached the solution
            if state[0] == frozenset() and not state[1]:
                print(f"Solution found with cost {cost}")
                return cost
            # skip processing if this state has been visited with a lower cost
            if state in visited and visited[state] <= cost:
                continue
            visited[state] = cost
            # find all neighboring states and their move costs
            for neighbor, move_cost in self.get_neighbors(state):
                next_cost = cost + move_cost
                # if this new state is not visited, or there is a cheaper way to get to it, add it to the queue
                if neighbor not in visited or visited[neighbor] > next_cost:
                    heappush(priority_queue, item: (next_cost, neighbor))
        return float('inf')
```

UI:

```python
def cross_time(self):  1 usage
    n = int(input("Enter the number of people crossing the bridge (up to 20): "))
    cross_time = []
    for i in range(n):
        time = int(input(f"Enter the crossing time for person {i + 1}: "))
        cross_time.append(time)
    self.bridge_graph = BridgeGraph(cross_time)
    self.bridge_graph.generate_all_states()
    print("Bridge problem initialized.")

def bridge_and_torch(self):  1 usage
    if not self.bridge_graph:
        print("Please initialize the bridge problem first.")
        return
    result = self.bridge_graph.solution()
    print(f"The minimal time for all to cross the bridge is {result} minutes.")
```